

A Critical Code Review of the Web Witchcraft and Wizardry Project

Edmund Mulligan

April 12, 2026

Abstract

Abstract goes here.

Contents

1	Introduction	2
2	Architecture and Design	2
3	HTML and CSS	2
4	Javascript	3
4.1	localStorage.js	3
4.2	populateLineNumbers.js	3
4.3	injectGlossaryPopover.js	3
4.4	utils.js, queryParameters.js and debug.js	4
5	Reflections	4
6	Conclusion	4
A	Javascript Assessment Criteria	6

1 Introduction

This essay reviews the codebase of the Web Witchcraft and Wizardry project, a web-based interactive learning platform designed to teach users about web development concepts through a magical theme. The review covers the architecture and design of the project, as well as an analysis of the HTML, CSS, and JavaScript code, with an emphasis on the latter. Reflections on the development process and potential areas for improvement are also discussed. The code is available for review on GitHub at

<https://github.com/Birkbeck2/puffin-web-project-edmundmulligan>. The deployed code can be viewed in GitHub Pages or at

<https://web.witchcraft-and-wizardry.school/>. Results of automated tests (along with design documentation) can be viewed at

<https://web.witchcraft-and-wizardry.school/diagnostics>.

2 Architecture and Design

The design of the appearance and layout of the website was documented in the previous essay (Mulligan, 2026), so will not be covered here, but the code has also been designed so as to be maintainable and extensible, with a modular structure that separates concerns (Dijkstra, 1982). This is a front-end focused project, so the codebase is primarily composed of HTML, CSS, and JavaScript files, with a clear separation between structure, presentation, and behaviour (Keith and Sambells, 2010). Following DevOps principles (Allman, 2018), the project utilises a build process to allow automated testing, as well as a version control system to manage changes and collaborate with other developers (should there be any in the future).

Several constraints have been imposed on the project, partly to comply with the academic requirements of this course, but also to keep the project focused and manageable.

- Use only vanilla JavaScript, HTML, and CSS, without any external libraries or frameworks.
- No use of generative AI or Large Language Models (LLMs) in the development process.
- No server side code or databases, as the project is focused on front-end development and client-side interactivity.
- Ensure that the website is fully accessible and responsive, adhering to best practices in web development.
- The project is developed by a single individual (myself).

3 HTML and CSS

Since the focus of this assignment is Javascript, the HTML and CSS code will not be analysed in depth, but some general observations can be made. The HTML code is well-structured and semantic, with appropriate use of headings, lists, and other elements to create a clear and logical document structure. The CSS code is organised into separate files for layout and components, with consistent naming conventions and a clear separation of concerns. The use of CSS variables allows for easy theming and customization of the website's appearance. The CSS has been completely refactored since the last assignment, to make it more maintainable and efficient. This includes keeping all global variables in definition files, using layers to control the cascade and specificity of styles, as well as removing redundant and unused styles. Nested selectors have been used where possible to improve readability. At most three CSS files need to be loaded on any page (globals.css, main.css and a page specific CSS file if necessary), which makes it less likely a specific CSS file will be accidentally missed when developing new pages.

The CSS code has been heavily influenced by tutorials from Kevin Powell (Powell, 2026b) and his new paid course (Powell, 2026a), which have provided valuable insights into modern CSS techniques and best practices.

4 Javascript

Arguably, there is rather more JavaScript in this project than is required for academic purposes, but this is because the project is designed to be a fully functional and interactive web application, rather than just a demonstration of specific concepts. The JavaScript code is organised into separate files for different components and features. The code follows best practices for readability and maintainability (McConnell, 2004), with consistent naming conventions, clear comments, and a focus on simplicity and clarity.

There is insufficient space in this essay to discuss all the JavaScript code in detail so this section highlights some of the more interesting and complex aspects. See Appendix A for a sample of examples of JavaScript code that demonstrate the use of various concepts and techniques required by this assessment.

4.1 `localStorage.js`

Arguably this is the most complex Javascript file in the codebase. It was written after a code review using CodeQL (GitHub, 2024) identified that using unencrypted local storage was a security risk. Arguably, no sensitive data is being stored so this is a false positive, but best practice is to eliminate security flaws before they become a problem (OWASP Foundation, 2010). It uses AES-GCM 256-bit encryption (National Institute of Standards and Technology, 2001) to encrypt data before storing it in `localStorage`, and decrypts it when retrieving. The encryption key is derived from a passphrase using PBKDF2 with a random salt and the encrypted data is stored as a JSON string containing the ciphertext and salt.

4.2 `populateLineNumbers.js`

This code is used to display javascript code snippets on the website with line numbers. It works by selecting all elements with the class `'code-snippet'`, splitting their text content into lines, and then creating a new HTML structure that includes both the line numbers and the code lines. The line numbers are generated dynamically based on the number of lines in the code snippet, and the original code is preserved in a separate element for styling purposes. This allows for separation of concerns between the structure of the code and its presentation, as well as making it easier to maintain and update the code snippets in the future. It also allows for the code snippets to be easily copied without including the line numbers, which can be a common issue when displaying code on websites.

4.3 `injectGlossaryPopover.js`

This code fetches glossary data from `glossary.html` and injects it into the page as popovers that appear when hovering over glossary terms. It works by selecting all elements with the class `'glossary-term'`, retrieving the corresponding definition from the glossary data, and then creating a popover element that is displayed on hover. To prevent injection attacks, the code sanitises the glossary definitions by escaping any HTML tags before inserting them into the popover. This ensures that any potentially malicious code in the glossary definitions cannot be executed when the popover is displayed. Rendering of the popover is handled by CSS, which, as usual, allows for separation of structure and presentation.

4.4 utils.js, queryParameters.js and debug.js

These scripts are not directly related to any specific features of the website, but provide utility functions and debugging tools that are used throughout the codebase. The `utils.js` file contains a variety of helper functions for tasks such as DOM manipulation, event handling, and data formatting. The `queryParameters.js` file provides a simple interface for working with URL query parameters, allowing for easy retrieval and manipulation of parameters in the URL. The `debug.js` file contains functions for logging and displaying debug information, which was essential during development and testing.

5 Reflections

6 Conclusion

Word count

Word count: 1080 words (excluding references)

References

- Eric Allman. Devops: Software architects' perspectives. *Communications of the ACM*, 61(4): 40–44, 2018. doi: 10.1145/3186331.
- Edsger W. Dijkstra. On the role of scientific thought. In *Selected Writings on Computing: A Personal Perspective*, pages 60–66. Springer, New York, 1982. Originally published in 1974. Introduces the principle of separation of concerns.
- GitHub. CodeQL: The code analysis engine for finding security vulnerabilities, 2024. URL <https://codeql.github.com/>. Accessed: 2026-04-12.
- Jeremy Keith and Jeffrey Sambells. *DOM Scripting: Web Design with JavaScript and the Document Object Model*. Apress, Berkeley, CA, 2nd edition, 2010. Advocates for unobtrusive JavaScript and separation of structure (HTML), presentation (CSS), and behavior (JavaScript).
- Steve McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, 2nd edition, 2004. ISBN 978-0735619678.
- Edmund Mulligan. A critical reflection on the design of the web witchcraft and wizardry project, 2026.
- National Institute of Standards and Technology. Advanced encryption standard (aes). Technical Report FIPS PUB 197, National Institute of Standards and Technology, Gaithersburg, MD, 2001. URL <https://doi.org/10.6028/NIST.FIPS.197>.
- OWASP Foundation. Secure coding practices - quick reference guide, 2010. URL <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>. Accessed: 2026-04-12.
- Kevin Powell. Css demystified. Online course, 2026a. URL <https://courses.thecascade.dev/course/css-demystified/>. Paid course. Accessed: April 2026.

Kevin Powell. Kevin powell youtube channel, 2026b. URL <https://www.youtube.com/@KevinPowell>. Accessed April 2026.

Appendices

A Javascript Assessment Criteria

Here are examples of Javascript concepts used in the codebase to allow assessors to easily identify where key features are implemented. This is not an exhaustive list of all Javascript concepts used in the codebase, but rather a quick guide to where an instance of each key feature can be found. Assessors are, of course, free to review the codebase in its entirety to gain a deeper understanding of the implementation and design choices made throughout the project.

Variables and Functions — `animatePortrait.js`

`setRandomPortraitPosition (container)` is a method (function) that uses variables to store the position of the portrait and the dimensions of the container. It also calls JavaScript's built-in `Math.random ()` function to generate random positions for the portrait.

Objects and Arrays — `animatePortrait.js`

`PortraitAnimator` class is instantiated as an object and uses an array to store the portrait images that are animated.

Conditional Logic and Loops — `animateWand.js`

Uses a `foreach` loop to iterate over path elements in `svg` base element and a conditional to deal with the possibility that the `svg` file does not contain a base `svg` element and to determine if a path element should be animated with a sparkle.

DOM Manipulation — `animatePortrait.js`

The DOM is manipulated in method `setPortraitImage (container, image)` to change the source of the portrait image and update the alt text.

Interactivity — `toggleSection.js`

`toggleSectionCollapse (sectionId, event)` is a method that toggles the collapse state of a section in response to user triggered events (e.g. clicking a button). It uses event listeners to detect user interactions and updates the DOM accordingly to show or hide the content of the section.

Data Retrieval and Storage — `localStorage.js`

The `LocalStorage` class provides methods for storing and retrieving encrypted data from the browser's local storage. This allows the application to persist user preferences or other data across sessions.